



COMSATS University Islamabad

Sahiwal Campus

Department of Computer Science

Course Title: Object Oriented Programming

Project Title: University Management System

Submitted To

Sir Ameer Ahmed Gillani

Submitted By (Group Members)

Umar Nadeem FA24-BSE-005

Arbab Jabeen FA24-BSE-006

Basmah Haroon FA24-BSE-007

Rabia Mukhtar FA24-BSE-021

Zuha Ishtiaq FA24-BSE-034

Program: BS Software Engineering (BSSE)

Semester: FA-24

Section: A

Session: 2024 – 2028

Table of Contents

Abstract.....	5
Introduction.....	6
Project Overview	6
Key Features	6
Project Objectives	8
System Architecture.....	8
Entity Relationship Diagram (ERD).....	10
Key Benefits.....	11
Technology Stack.....	11
Frontend (User Interface).....	11
Backend (Business Logic)	12
Database (Data Storage)	12
Libraries Used.....	13
Development Environment	14
System Design	14
UI Design.....	14
Database Design.....	19
Class Diagram.....	22
Flowchart	23
Functionalities.....	25
Admin Login.....	25
Add/Update/View Student/Teacher Details.....	27
Marks Entry	27
Leave Management.....	28
Fee Structure & Fee Form.....	28
Examination Details.....	29
About Section.....	29
Database Schema	29
Tables.....	30
Relationships.....	33
Data Integrity and Constraints	35
Installation Guide.....	35
Prerequisites.....	35
Steps to Set Up.....	36

Running the Application	38
Troubleshooting	38
User Manual.....	39
Admin Login.....	39
Navigating the System	40
Adding/Updating/View Student/Teacher Details	40
Marks Entry	41
Leave Management.....	41
Fee Structure & Fee Form.....	42
Examination Details.....	42
About Section.....	43
Future Enhancements.....	49
Role-Based Access Control	49
Mobile Application	50
Data Analytics and Reporting	50
Integration with Other Systems	51
Cloud Deployment	51
Enhanced User Interface	52
AI Integration for Predictive Analytics.....	52
Multi-Language Support.....	53
Conclusion	53
Final Thoughts	54

Table of Figures

Table 1 Manual Project Screenshots	43
--	----

Figures

Figure 1 ER-Diagram.....	10
Figure 2 Login.....	15
Figure 3 Sidebar.....	15
Figure 4 Teacher Form	16
Figure 5 Student Form	17
Figure 6 Teacher Details	18
Figure 7 Student Details.....	18
Figure 8 Teacher's leave details	19
Figure 9 Student's leave details.....	19
Figure 10 Class-Diagram	23
Figure 11 FlowChart	24
Figure 12 Invalid Login	26
Figure 13 Valid Login	26
Figure 14 ER-Diagram.....	34

Abstract

This project presents the design and implementation of a robust **University Management System (UMS)** developed in **Java** utilizing the **Swing framework** for a rich Graphical User Interface (GUI) and **MySQL** as the persistent database backend. The system is engineered to efficiently manage critical university operations, including detailed management of student and faculty records, a streamlined leave application and tracking system, and essential administrative functionalities such as searching, updating, printing, and generating reports. The core objective is to replace error-prone manual processes with an integrated, automated solution, ensuring data integrity and improving administrative efficiency. The application's modular design, leveraging the **Model-View-Controller (MVC) conceptual pattern** for structure, was rigorously tested and validated to ensure reliability and user-friendliness.

Introduction

Project Overview

The **University Management System (UMS)** is a software solution designed to streamline the administrative tasks of managing students, teachers, and various university-related activities. Built using **Java** and **MySQL**, this system provides an easy-to-use platform for university administrators to manage academic, financial, and leave-related information. The system offers an efficient way to handle large volumes of data and automate routine tasks such as recording student marks, managing teacher and student details, processing leave applications, and maintaining fee structures.

The system is designed with a **Graphical User Interface (GUI)** using **Java Swing**, allowing for easy navigation and a user-friendly experience. With proper data validation, secure login mechanisms, and integrated reports generation, UMS aims to significantly reduce the administrative workload and ensure data integrity and security.

Key Features

The **University Management System** includes several key features that enhance the user experience and provide functionality for managing various aspects of university administration. These features include:

1. Admin Login & Security:

- A secure **admin login page** ensures that only authorized users can access and manage the system's data.
- The system uses **username** and **password** for authentication, ensuring that unauthorized access is prevented.

2. Sidebar Navigation:

- Once logged in, the user is presented with a **sidebar menu**, offering quick access to the system's core features, such as:
 - **Add Student / Teacher:** Add new students and teachers to the system with necessary details (e.g., name, contact, academic information).
 - **Update Student / Teacher:** Modify existing student or teacher records.
 - **View Student / Teacher Details:** View records for all students and teachers stored in the system.
 - **Marks Entry:** Enter and update student marks for various subjects.

- **Fee Management:** View and update fee structures for students and generate student fee forms.
- **Leave Management:** Manage leave requests for both students and teachers, ensuring proper record-keeping of absences.
- **Examination Details:** Maintain and display details regarding university exams, including schedules, subjects, and examiners.
- **About Section:** Display the project's group members and provide additional information about the system.

3. **Student and Teacher Management:**

- **Add Student / Teacher:** The system allows administrators to add new student and teacher records with necessary personal and academic details (such as ID, name, department, and contact).
- **Update Student / Teacher:** Update records for existing students and teachers, including their contact details, academic information, and leave records.
- **View Student / Teacher Details:** View a comprehensive list of students and teachers, including their marks, fee status, leave records, and personal details.

4. **Marks Entry:**

- **Enter Marks:** Admins can enter marks for students for various subjects. The system provides a dedicated form for inputting marks, validating them, and ensuring that data is entered correctly.
- **View Marks:** Admins can view the marks entered for each student, organized by subject and exam.

5. **Leave Management:**

- **Student Leave:** The system allows administrators to track student leave records, including dates and reasons for absence. Admins can add or modify leave records as necessary.
- **Teacher Leave:** Similar to student leave management, the system tracks teacher leave records, ensuring that leave policies are adhered to and teaching schedules are updated accordingly.

6. **Fee Structure & Fee Form:**

- **Fee Structure Management:** The admin can update the university's fee structure, ensuring that the latest fee information is always available.

- **Fee Form Generation:** A feature that allows the admin to generate fee forms for students, reflecting the current fee structure.

7. **Examination Details:**

- Admins can add, view, or modify details about upcoming examinations. This includes information like exam schedules, subjects, and examiners.

8. **About Section:**

- Provides information about the team involved in the development of the system. It includes the names of the project members, their roles, and other relevant details.

Project Objectives

The primary objective of the **University Management System** is to provide an efficient, user-friendly, and secure platform to handle all administrative tasks in a university. The objectives of the system include:

1. **Simplifying Administrative Tasks:**

- By automating the management of student and teacher details, marks entry, leave records, and fee structures, the system reduces the administrative burden and allows staff to focus on other critical tasks.

2. **Ensuring Data Accuracy and Integrity:**

- The system enforces proper validation rules and constraints, such as checking for valid student IDs, ensuring marks fall within acceptable ranges, and verifying the accuracy of entered data.

3. **Secure Data Management:**

- The system ensures that all sensitive data, including personal details and academic records, are securely stored and protected from unauthorized access.

4. **Facilitating Reporting:**

- The system provides functionality to generate reports related to student performance, leave records, and fee payments, allowing for better decision-making and record-keeping.

System Architecture

The **University Management System** follows a **three-tier architecture**:

1. **Presentation Layer (UI):**

- Built using **Java Swing**, the presentation layer provides a GUI that allows the user to interact with the system. It consists of login screens, data entry forms, and result screens.

2. **Business Logic Layer (Backend):**

- The core of the system, where the Java classes implement the business logic. This layer manages operations such as data validation, calculations, and logic for adding/updating records.

3. **Data Layer (Database):**

- The **MySQL database** serves as the data layer, where all the system's data (student details, marks, teacher information, etc.) are stored and retrieved.

Entity Relationship Diagram (ERD)

This Entity Relationship Diagram (ERD) represents the structural design of the University Management System database. It illustrates the entities, their attributes, and the relationships among students, teachers, courses, fees, subjects, and academic records. The diagram provides a clear understanding of how data is organized and interconnected within the system.

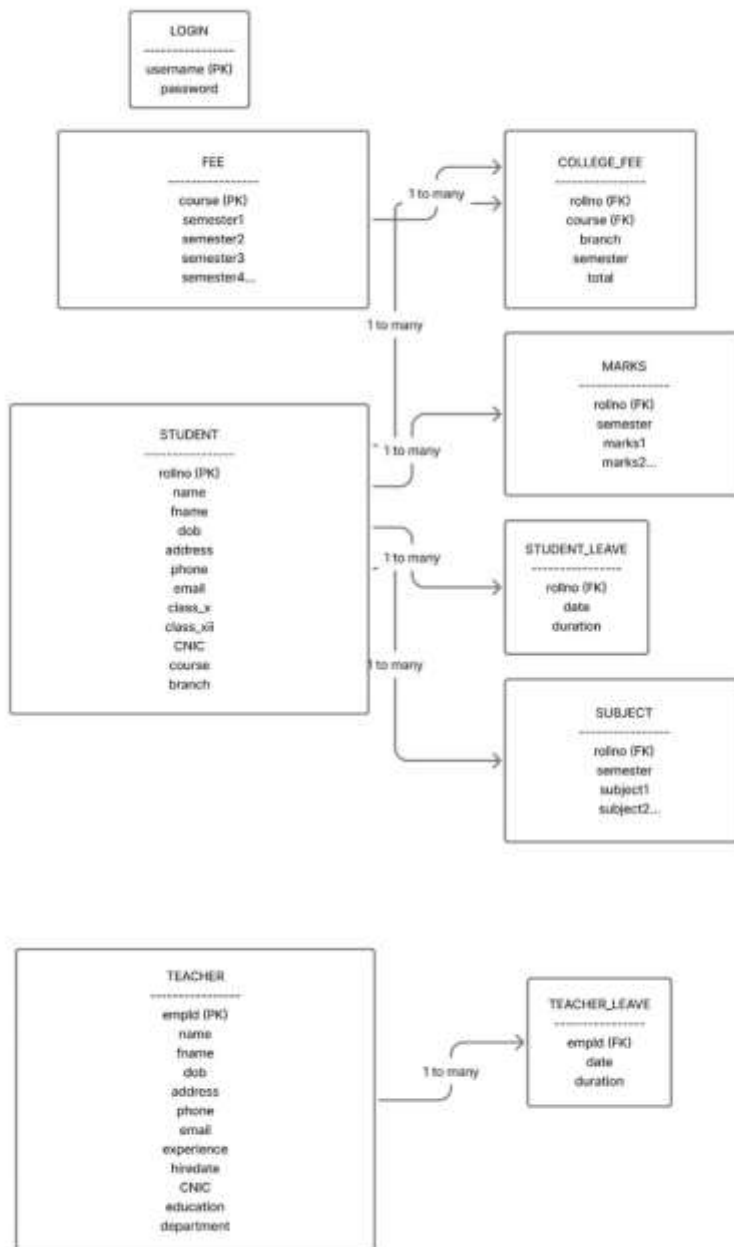


Figure 1 ER-Diagram

Key Benefits

1) **Efficiency:**

The system automates repetitive tasks such as student record entry, marks entry, and leave management, improving the efficiency of administrative staff.

2) **Data Security:**

The use of secure login and validation mechanisms ensures that sensitive data is protected, and only authorized personnel can access the system.

3) **Scalability:**

The system is designed to handle increasing numbers of students and teachers, making it scalable for large institutions.

4) **Flexibility:**

The system is designed to accommodate changes in the university's policies, such as modifications to the fee structure, leave policies, or marks entry processes.

Technology Stack

The **University Management System** is developed using a combination of **Java** for the backend logic and **MySQL** for data management. Below is an in-depth overview of the technologies used to build this system:

Frontend (User Interface)

- **Java Swing:**

The user interface (UI) of the University Management System is built using **Java Swing**, which is a part of the Java Foundation Classes (JFC). Java Swing allows for the development of graphical user interfaces with lightweight components such as buttons, labels, tables, and forms. Swing is highly customizable and platform-independent, making it an ideal choice for this project.

Key Features of Java Swing in this Project:

- **Main Window and Sidebar Navigation:** The main window includes a sidebar for easy access to all system functionalities (e.g., student management, marks entry, fee structure, etc.).
- **Forms and Dialogs:** Custom forms are used to input student, teacher, and exam details, while dialog boxes help display messages or alerts to the user.

- **Tables:** Used for displaying student records, marks, teacher details, and leave records in an organized manner.

Backend (Business Logic)

- **Java:**

The core logic of the system is implemented in **Java**, which is used for handling the business rules and data processing. Java provides robust tools and libraries for exception handling, object-oriented programming (OOP), and multithreading.

The backend handles key functionalities such as:

- User authentication and session management.
- CRUD operations (Create, Read, Update, Delete) for students, teachers, marks, and leave records.
- Validation of input data such as ensuring marks are within acceptable ranges, phone numbers are valid, and IDs are unique.

Key Java Features Used:

- **Swing Event Handling:** For managing user interactions with the interface (e.g., button clicks, form submissions).
- **File I/O:** For saving or exporting reports.
- **Data Validation:** Ensures all input data is valid and consistent with the system's constraints (e.g., no negative marks, no duplicate student IDs).
- **Error Handling:** Implemented using Java's try-catch blocks to gracefully handle runtime exceptions.

Database (Data Storage)

- **MySQL:**

The system uses **MySQL** as the relational database management system (RDBMS) to store data such as student and teacher records, marks, leave records, and fee structures. MySQL is known for its speed, reliability, and scalability, making it a suitable choice for handling large volumes of data in an educational institution.

Key Features of MySQL in this Project:

- **Relational Database Structure:** The data is organized into multiple tables such as students, teachers, marks, leave_records, and fees.

- **SQL Queries:** The system uses SQL queries to perform CRUD operations on the database. It retrieves and updates student and teacher data, marks, leave records, and examination details.
- **Data Integrity:** Data integrity is ensured by enforcing primary key constraints, foreign key constraints, and data type consistency (e.g., integer for marks, varchar for names).

Key MySQL Features Used:

- **Stored Procedures:** For frequently executed operations such as inserting new records or updating data.
- **Joins:** Used to combine data from multiple tables (e.g., fetching student and teacher details together).
- **Transactions:** Ensures that multiple database operations, like adding marks or updating leave records, are executed atomically.

Libraries Used

The system uses the following libraries to enhance functionality and user experience:

1. **mysql-connector-java (JDBC Driver):**

- The **JDBC** (Java Database Connectivity) driver enables the system to interact with the MySQL database. It facilitates the communication between the Java backend and the MySQL database to execute queries and retrieve data.

2. **JCalendar (jcalendar-tz-1.3.3-4.jar):**

- This library provides calendar components for Java Swing, allowing the user to select dates for events such as examination dates, leave requests, and fee payment deadlines.

3. **FlatLaf (flatlaf-3.7.jar):**

- **FlatLaf** is a modern Java Look and Feel (L&F) library that provides a sleek, minimalist user interface. This is used to enhance the UI's aesthetic appeal and make it more visually appealing to the user.

4. **RS2XML (rs2xml.jar):**

- This library helps in converting **ResultSet** (from MySQL queries) into **JTable** models for easy display of database query results in a table format on the GUI.

Development Environment

- **IDE Used:**

The project is developed using **NetBeans IDE**, which supports Java development and provides features such as debugging, GUI design, and code refactoring. It allows for easy integration with MySQL and efficient management of project files.

- **Version Control:**

The system uses **Git** for version control, enabling collaboration among team members and tracking changes made to the project.

System Design

The **System Design** section provides a detailed overview of how the **University Management System** is structured and how different components of the system work together. This section covers the design of the **user interface (UI)**, the **database structure**, and the overall architecture of the system. It also includes class and flow diagrams to visually represent the components and their interactions.

UI Design

The **User Interface (UI)** is developed using **Java Swing**, which provides a variety of lightweight components to build the graphical user interface. The UI is designed to be intuitive and easy to navigate, ensuring that users can quickly access all the necessary functionalities.

Key UI Components:

- **Login Page:**

- A login page allows only authorized personnel (admin) to access the system. The admin must enter a valid username and password to proceed.

❖ Login Page Screenshot

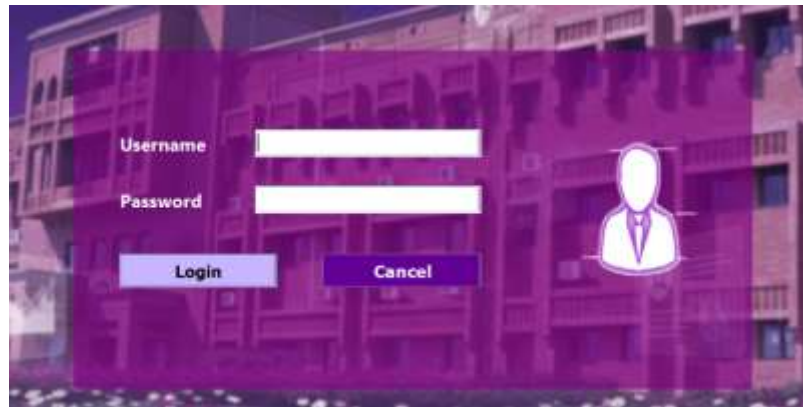


Figure 2 Login

- **Sidebar Navigation:**

- The main interface features a sidebar navigation menu, offering easy access to different sections such as **Student Management**, **Marks Entry**, **Fee Management**, **Leave Management**, and **Examination Details**.

- ❖ **Sidebar Menu Screenshot**



Figure 3 Sidebar

- **Forms for Data Entry:**

- The system uses various forms to add or update student, teacher, marks, and fee records. These forms include input fields for all relevant details, with built-in validations to ensure correct data entry.

❖ **Form Screenshot for Adding Student or Teacher**

New Teacher Details			
Name		Father's Name	
Employee Id	1019096	Date of Birth	
Address		Phone	
Email Id		Hire Date	
Exerience	1	CNIC Number	
Qualification	B.Tech	Department	Computer Science
Submit		Cancel	

Figure 4 Teacher Form

New Student Details

Name		Father's Name	
Roll Number	15336182	Date of Birth	
Address		Phone	
Email Id		Class X (%)	
Class XII (%)		CNIC Number	
Course	B.Tech	Branch	Computer Science
Submit		Cancel	

Figure 5 Student Form

- **Tables for Data Display:**

- Data such as student records, marks, and leave records are displayed in tables with sorting and filtering options. This makes it easy for the admin to view and manage large datasets.

- ❖ **Table View Screenshot**



Figure 6 Teacher Details

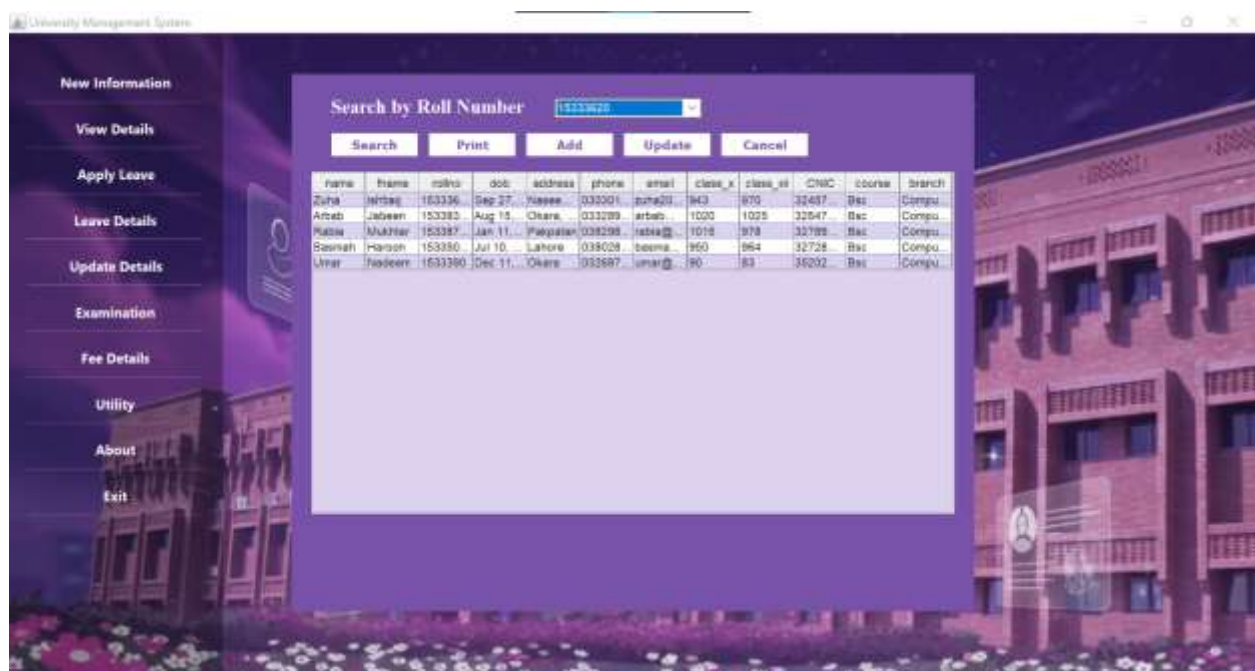


Figure 7 Student Details

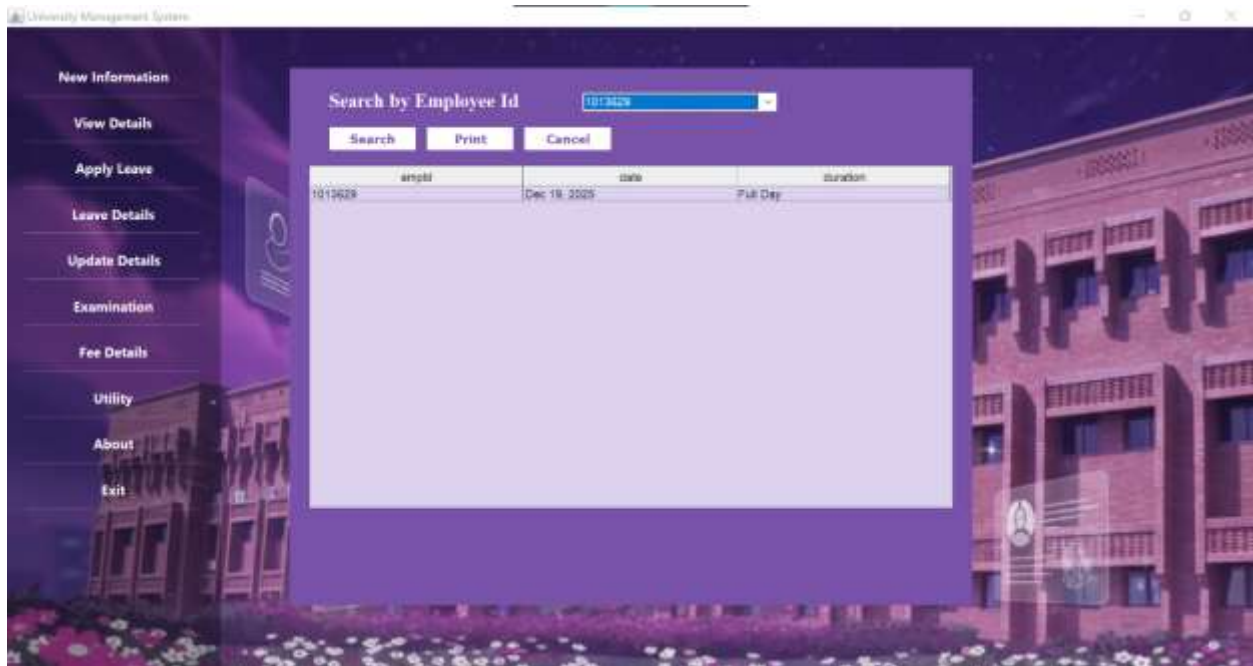


Figure 8 Teacher's leave details

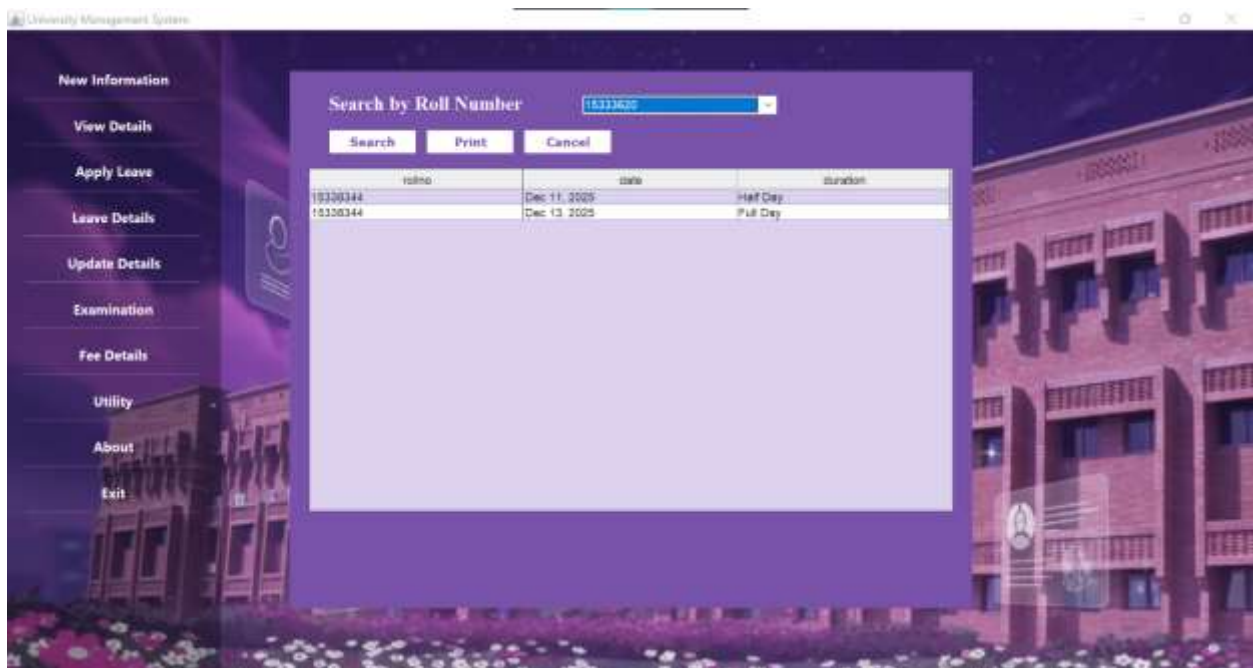


Figure 9 Student's leave details

Database Design

The **University Management System** uses **MySQL** to store data related to students, teachers, marks, leave records, and more. The database is designed using a **relational model**, where

different entities are represented by tables, and relationships between entities are established through foreign keys.

Key Database Tables:

1. Students Table:

- Stores details such as **student_id**, **name**, **address**, **phone_number**, **email**, **department**, and **marks**.

Example Schema:

```
CREATE TABLE students (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(100),  
    address VARCHAR(255),  
    phone_number VARCHAR(15),  
    email VARCHAR(100),  
    department VARCHAR(50),  
    marks DECIMAL(5, 2)  
);
```

2. Teachers Table:

- Stores teacher-related information such as **teacher_id**, **name**, **subject**, **email**, and **phone_number**.

Example Schema:

```
CREATE TABLE teachers (  
    teacher_id INT PRIMARY KEY,  
    name VARCHAR(100),  
    subject VARCHAR(50),  
    email VARCHAR(100),  
    phone_number VARCHAR(15)  
);
```

3. Marks Table:

- Stores marks for students, with references to the **students** table.

Example Schema:

```
CREATE TABLE marks (  
    mark_id INT PRIMARY KEY,  
    student_id INT,  
    subject VARCHAR(50),  
    marks DECIMAL(5, 2),  
    FOREIGN KEY (student_id) REFERENCES students(student_id)  
);
```

4. Leave Records Table:

- Keeps track of student and teacher leave records.

Example Schema:

```
CREATE TABLE leave_records (  
    leave_id INT PRIMARY KEY,  
    person_id INT,  
    leave_type VARCHAR(50),  
    leave_date DATE,  
    reason TEXT,  
    FOREIGN KEY (person_id) REFERENCES students(student_id) OR FOREIGN KEY  
(person_id) REFERENCES teachers(teacher_id)  
);
```

5. Fee Structure Table:

- Stores details of the fee structure.

Example Schema:

```
CREATE TABLE fee_structure (  
    fee_id INT PRIMARY KEY,  
    department VARCHAR(50),
```

```
fee_amount DECIMAL(10, 2)
);
```

6. Examination Details Table:

- Stores information about exams, such as **exam_id**, **subject**, **date**, and **time**.

Example Schema:

```
CREATE TABLE examination_details (
    exam_id INT PRIMARY KEY,
    subject VARCHAR(50),
    exam_date DATE,
    exam_time TIME
);
```

Class Diagram

The **Class Diagram** represents the static structure of the University Management System, showing the main classes and their relationships.

- **Student Class:**
 - Represents the **students** entity. Contains attributes such as **student_id**, **name**, **address**, **marks**, and methods to **add**, **update**, or **retrieve** student information.
- **Teacher Class:**
 - Represents the **teachers** entity. Contains attributes like **teacher_id**, **name**, **subject**, and methods for managing teacher data.
- **Leave Class:**
 - Represents the leave records for both students and teachers. It contains attributes like **leave_id**, **leave_type**, and methods to manage leave records.
- **Marks Class:**
 - Responsible for managing student marks. Includes attributes like **mark_id**, **subject**, and methods to **add** or **modify** marks.

❖ Class Diagram

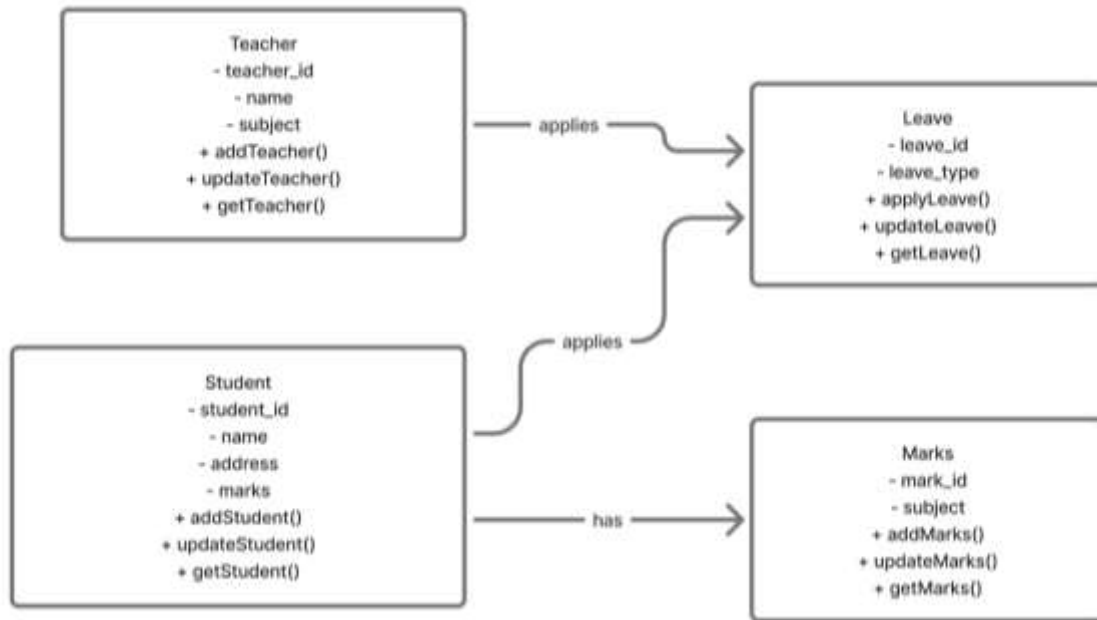


Figure 10 Class-Diagram

Flowchart

The **Flowchart** below outlines the process flow for the **Admin Login** functionality, which is one of the critical processes in the system.

- 1) **Start**
- 2) **Admin enters credentials (username and password)**
- 3) **Validate credentials**
 - a. If valid → Proceed to Main Dashboard
 - b. If invalid → Display error message
- 4) **End**

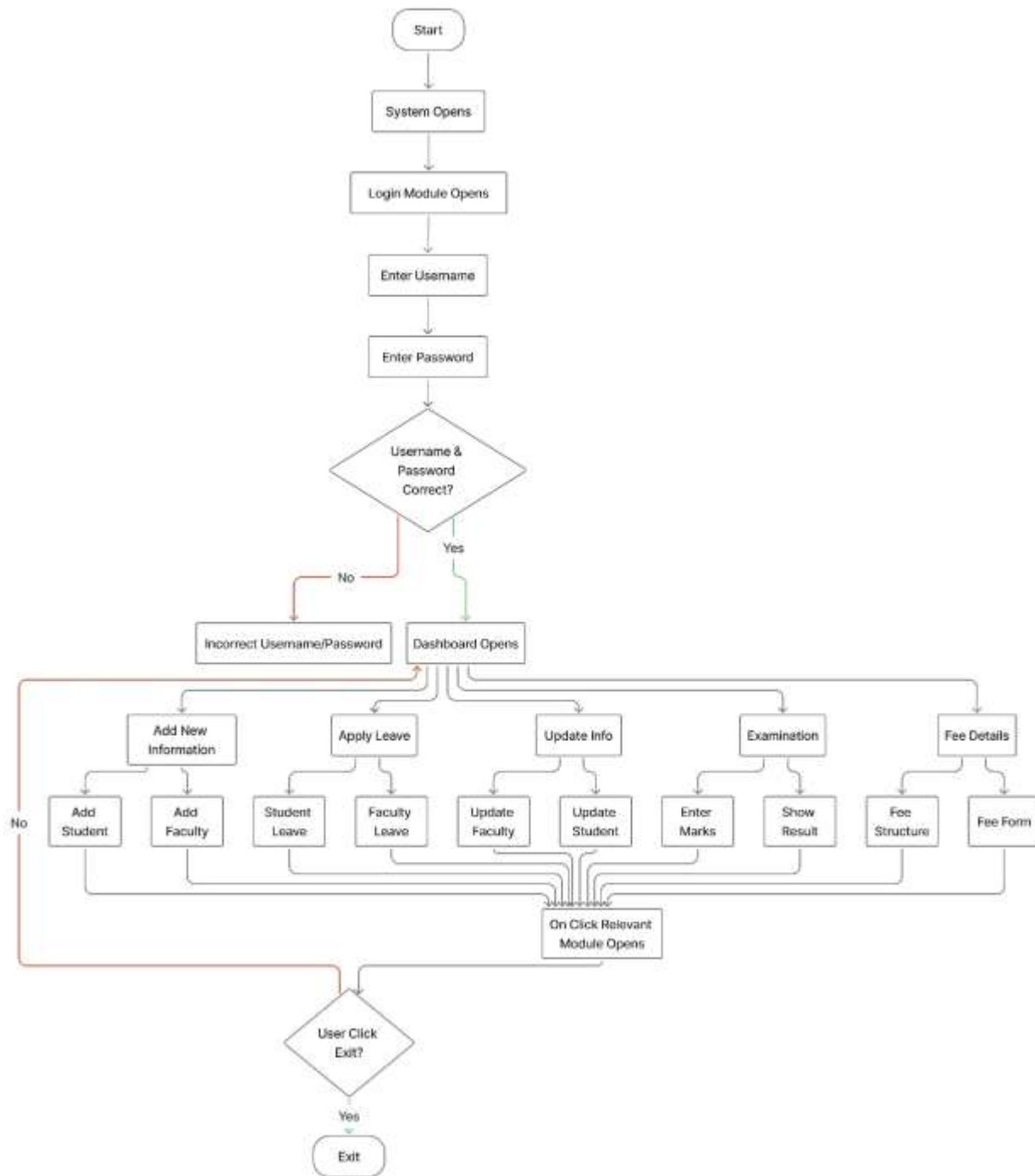


Figure 11 FlowChart

Functionalities

This section outlines the key functionalities of the **University Management System**. Each functionality is described in detail, highlighting how the system works, the user interactions, and how the business logic is implemented. This section also includes examples of how each feature operates.

Admin Login

The **Admin Login** page is the first screen that the user encounters when accessing the system. It ensures that only authorized users can access the system's features.

- **Functionality:**

- The login page consists of two input fields: one for the **username** and one for the **password**.
- **Input Validation:** Both fields are required to be filled before the system can attempt to authenticate the user. The password is masked for security purposes.
- **Authentication:** Upon submission, the system checks the provided credentials against the **admin table** in the database. If the credentials match, the admin is granted access to the main dashboard. If the credentials are invalid, an error message is displayed.

- **Flow:**

1. The admin enters the username and password.
2. The system validates the input fields.
3. The credentials are checked against the database.
4. If the credentials are valid, the admin is redirected to the main dashboard; otherwise, an error message is shown.

❖ **Login Page Screenshot**

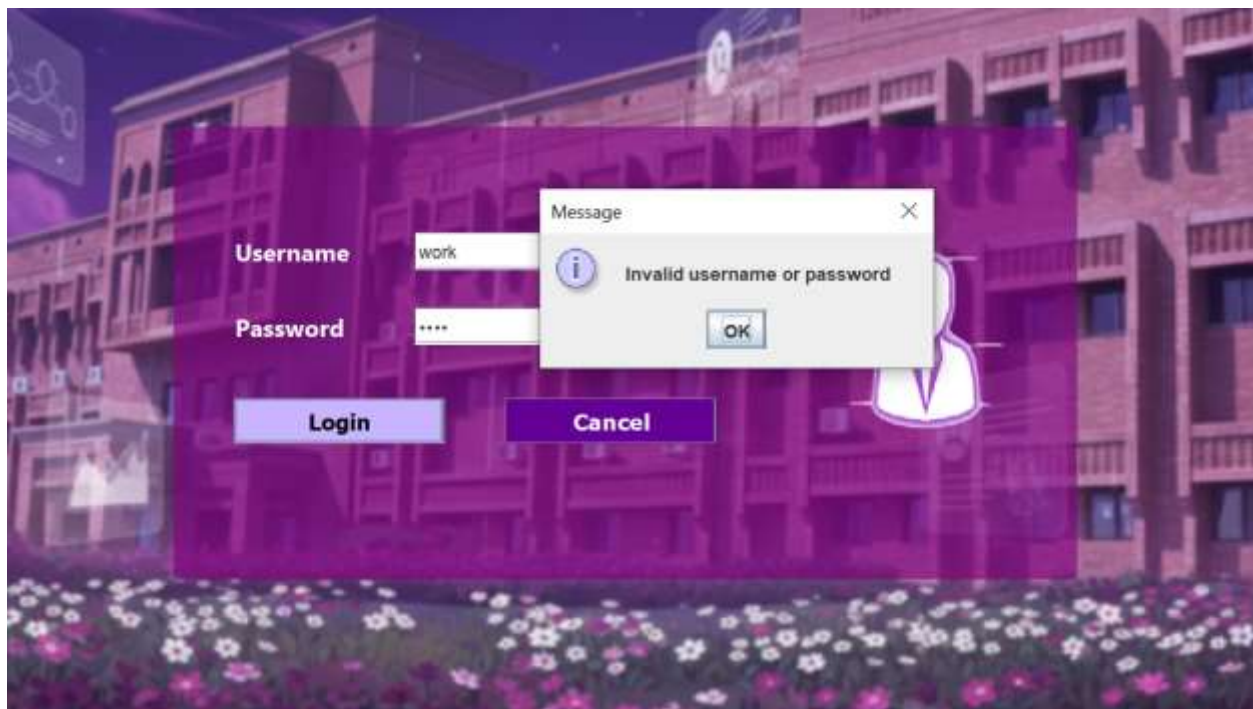


Figure 12 Invalid Login

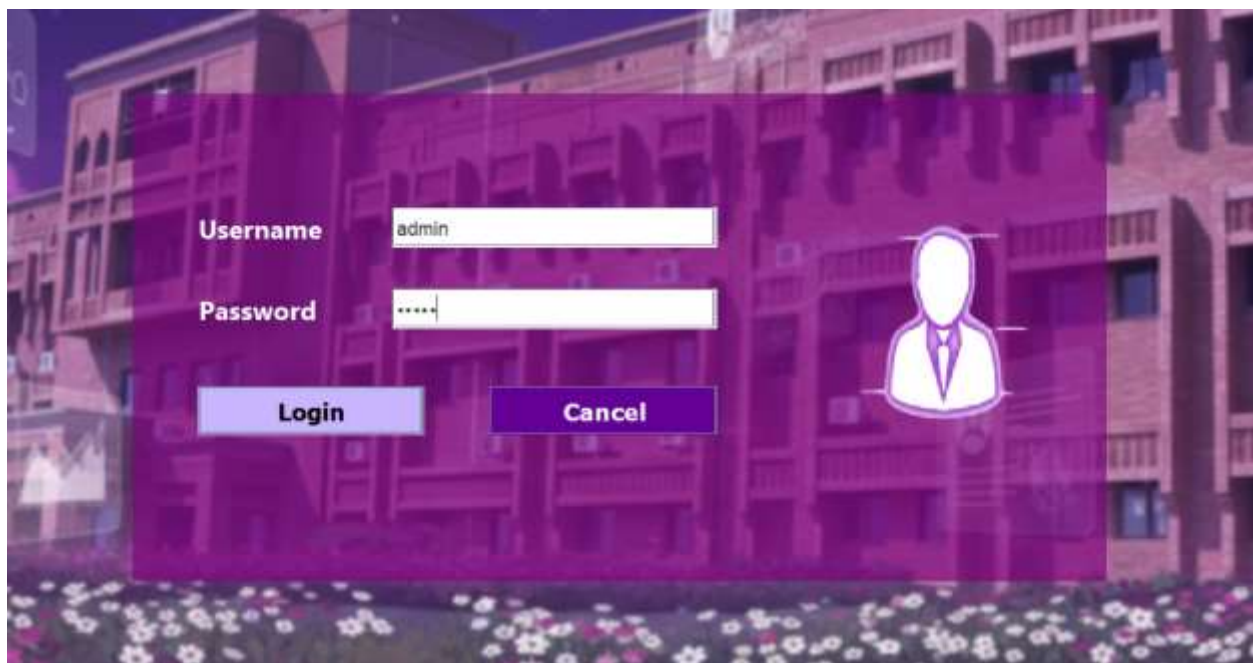


Figure 13 Valid Login

Add/Update/View Student/Teacher Details

The system allows the **admin** to manage both **student** and **teacher** records. These records can be added, updated, or viewed in a detailed format.

- **Add Student/Teacher:**
 - The admin can add new students or teachers by filling in a form with relevant details such as name, department, contact information, and academic details.
 - **Validation:** The system checks for duplicate entries (e.g., duplicate student IDs or email addresses) before saving the data to the database.
- **Update Student/Teacher:**
 - The admin can modify the details of an existing student or teacher by selecting the record from a list and editing the required fields.
 - **Validation:** The system ensures that the updated information is valid (e.g., email format, phone number format).
- **View Student/Teacher:**
 - The admin can view all stored records in a tabular format. The records are displayed with pagination, making it easy to browse through large datasets.
 - **Search and Filter:** The records can be filtered by attributes like name, department, or student ID.

Marks Entry

This functionality allows the **admin** to enter and manage students' marks.

- **Functionality:**
 - The admin selects a student from the list and enters the marks for the relevant subjects.
 - **Validation:** The system ensures that marks are within a valid range (e.g., 0 to 100).
 - The marks are saved to the **marks table** in the database, where they are associated with the student and subject.

- **Flow:**

1. The admin selects a student.

2. The system presents a form to enter the marks for each subject.
3. After submitting the marks, the data is validated and stored in the database.
4. The system updates the student's record with the new marks.

Leave Management

The system allows the admin to manage leave records for both students and teachers.

- **Student Leave:**
 - The admin can add, update, and view leave records for students. A student can request leave for a particular period, and the admin approves or rejects the request.
 - **Functionality:** The leave details (e.g., leave date, reason) are stored in the database. The system tracks leave history for each student.
- **Teacher Leave:**
 - Similar to student leave, the admin can manage teacher leave requests. Teachers can request leave, and the admin can approve or deny the request.
 - **Flow:** The system verifies that the teacher has not exceeded the allowed number of leave days.
- **Leave Report:**
 - The admin can view a report of all leave records for students or teachers. The report displays leave dates, reasons, and the status of each leave request (approved or denied).

Fee Structure & Fee Form

This functionality enables the admin to manage the university's fee structure and generate fee forms for students.

- **Fee Structure Management:**
 - The admin can add, update, and delete the fee structure for different departments.
 - **Functionality:** The system ensures that the fee amounts are updated and displayed accurately for each department.
- **Fee Form Generation:**

- The admin can generate fee forms for students. This form includes all the necessary fee details (tuition fees, lab fees, etc.), along with the student's information.
- **Flow:** The student's fee form can be printed or saved as a PDF.

Examination Details

This functionality allows the admin to manage examination details, such as the exam schedule, subjects, and examiners.

- **Add Examination Details:**
 - The admin can add new examination details, including the subject, date, and time.
 - **Flow:** Once the exam details are submitted, the system stores them in the **examination_details** table in the database.
- **View Examination Details:**
 - The admin can view all upcoming exams in a table, sorted by date or subject.
 - **Search and Filter:** The admin can search for exams based on the subject or date.

About Section

This section provides information about the project team and offers credit to the developers of the system.

- **Content:**
 - The **about page** contains the names and roles of the group members who contributed to the project. It may also include additional details such as project goals, objectives, and technologies used.

Database Schema

The **Database Schema** section outlines the structure of the database, including the tables, fields, relationships, and how data is organized within the **University Management System**. The schema is designed to store information related to students, teachers, marks, leave records, fee structures, and examination details.

The database follows a **relational model**, ensuring that data is stored in a structured manner with clear relationships between different entities.

Tables

The database is divided into several key tables, each designed to store specific information. Below is an overview of the primary tables used in the **University Management System**:

1. Students Table:

- Stores all details related to students, including personal information, contact details, and academic records.

```
CREATE TABLE students (  
    student_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    address VARCHAR(255),  
    phone_number VARCHAR(15),  
    email VARCHAR(100) NOT NULL UNIQUE,  
    department VARCHAR(50),  
    marks DECIMAL(5, 2)  
);
```

Key Fields:

- student_id: Unique identifier for each student.
- name: Student's full name.
- address: Student's home address.
- phone_number: Contact number of the student.
- email: Email address, must be unique.
- department: Department to which the student belongs.
- marks: The marks scored by the student in exams.

2. Teachers Table:

- Stores information about the teachers.

```
CREATE TABLE teachers (
```

```

teacher_id INT PRIMARY KEY AUTO_INCREMENT,
name VARCHAR(100) NOT NULL,
subject VARCHAR(50),
email VARCHAR(100) NOT NULL UNIQUE,
phone_number VARCHAR(15)
);

```

Key Fields:

- teacher_id: Unique identifier for each teacher.
- name: Teacher's full name.
- subject: Subject taught by the teacher.
- email: Email address, must be unique.
- phone_number: Contact number of the teacher.

3. Marks Table:

- Stores the marks for each student in various subjects.

```

CREATE TABLE marks (
mark_id INT PRIMARY KEY AUTO_INCREMENT,
student_id INT,
subject VARCHAR(50),
marks DECIMAL(5, 2),
FOREIGN KEY (student_id) REFERENCES students(student_id)
);

```

Key Fields:

- mark_id: Unique identifier for each marks record.
- student_id: Reference to the student who received the marks.
- subject: Subject for which the marks are recorded.
- marks: Marks obtained by the student.

4. Leave Records Table:

- Tracks the leave records for students and teachers.

```
CREATE TABLE leave_records (
    leave_id INT PRIMARY KEY AUTO_INCREMENT,
    person_id INT,
    leave_type VARCHAR(50),
    leave_date DATE,
    reason TEXT,
    FOREIGN KEY (person_id) REFERENCES students(student_id)
    ON DELETE CASCADE
);
```

Key Fields:

- leave_id: Unique identifier for each leave record.
- person_id: References either a student or a teacher. A foreign key can reference the students or teachers table.
- leave_type: Type of leave (e.g., sick leave, vacation).
- leave_date: Date the leave was taken.
- reason: Reason for the leave.

5. Fee Structure Table:

- Contains the fee details for various departments.

```
CREATE TABLE fee_structure (
    fee_id INT PRIMARY KEY AUTO_INCREMENT,
    department VARCHAR(50),
    fee_amount DECIMAL(10, 2)
);
```

Key Fields:

- fee_id: Unique identifier for each fee record.
- department: Department to which the fee structure applies (e.g., Engineering, Arts).

- fee_amount: The fee amount for the department.

6. Examination Details Table:

- Stores information about examinations, including subject, date, and time.

```
CREATE TABLE examination_details (  
    exam_id INT PRIMARY KEY AUTO_INCREMENT,  
    subject VARCHAR(50),  
    exam_date DATE,  
    exam_time TIME  
);
```

Key Fields:

- exam_id: Unique identifier for each exam record.
- subject: Subject for the exam.
- exam_date: Date when the exam will take place.
- exam_time: Time when the exam will start.

Relationships

The **University Management System** uses relational integrity to establish connections between various entities. The key relationships between tables are as follows:

1. Student to Marks:

- A **one-to-many** relationship between the **students** and **marks** tables. Each student can have multiple records of marks, but each record is associated with one student.

2. Student to Leave Records:

- A **one-to-many** relationship between the **students** and **leave_records** tables. A student can have multiple leave records.

3. Teacher to Leave Records:

- A **one-to-many** relationship between the **teachers** and **leave_records** tables. A teacher can have multiple leave records.

4. Student to Fee Structure:

- Although the **fee_structure** table is not directly linked to the **students** table, the fee amount is associated with the student's department. This is handled by the department field in both tables.

5. Student to Examination Details:

- A **many-to-many** relationship exists between students and examinations, where each student can take multiple exams. However, this is typically handled through a linking table that connects students to their exams.

❖ ER Diagram or Database Schema Diagram

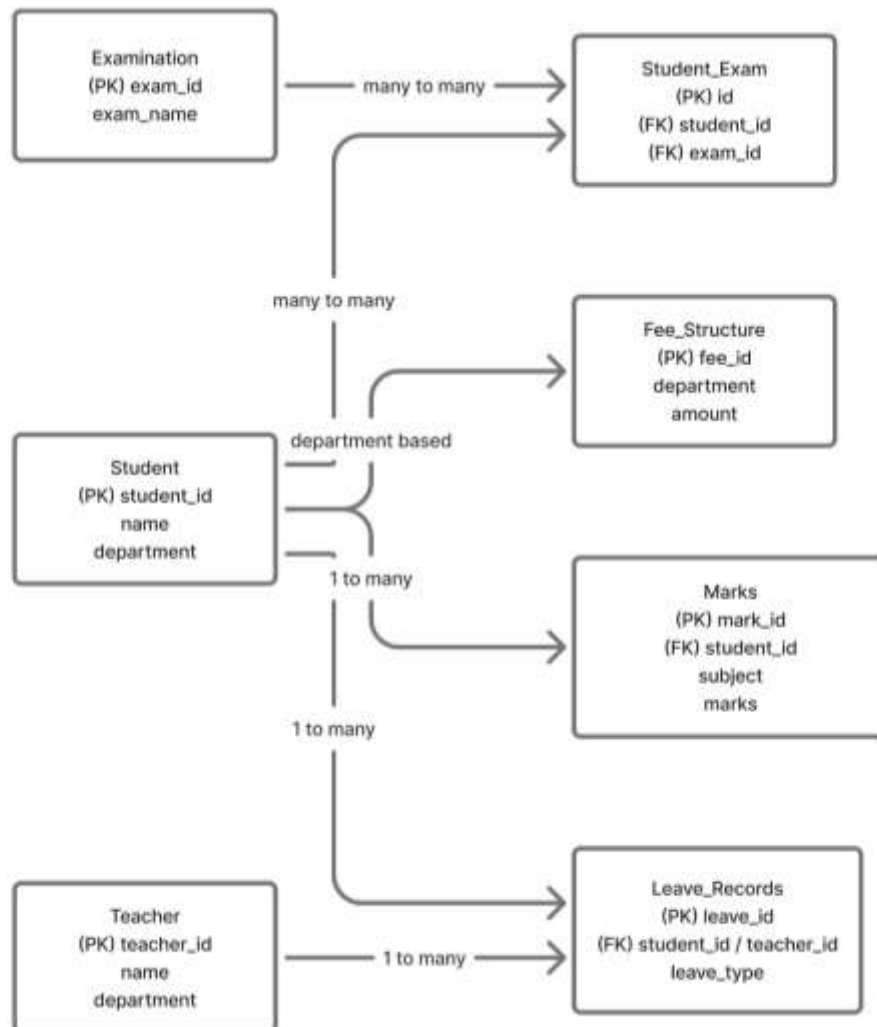


Figure 14 ER-Diagram

Data Integrity and Constraints

To ensure that the data is valid and consistent, the following constraints are applied:

- **Primary Key Constraints:**
Each table has a primary key that uniquely identifies each record (e.g., `student_id`, `teacher_id`, `mark_id`).
- **Foreign Key Constraints:**
Relationships between tables are maintained using foreign keys (e.g., `student_id` in the **marks** and **leave_records** tables). These ensure referential integrity, meaning that the data remains consistent between related tables.
- **Unique Constraints:**
Fields like **email** in the **students** and **teachers** tables are constrained to be unique, preventing duplicate entries.
- **Not Null Constraints:**
Important fields like **name**, **department**, and **marks** are marked as NOT NULL, ensuring that they are always filled out.
- **Check Constraints:**
The system ensures that **marks** fall within a valid range (e.g., between 0 and 100) using appropriate checks.

Installation Guide

This section provides detailed instructions on how to set up and run the **University Management System** on your local machine. The guide covers the necessary prerequisites, installation steps, and configuration needed to get the system up and running.

Prerequisites

Before installing and running the **University Management System**, ensure that the following software is installed on your system:

1. **Java Development Kit (JDK) 25 or above**
 - The project is developed using Java, and you need the **JDK** to compile and run the application.
 - You can download the JDK from the official [Oracle website](https://www.oracle.com/in/java/technologies/javase-downloads.html).

2. MySQL Database Server

- The system uses **MySQL** to manage and store data. You will need to install and configure **MySQL** on your machine or use a remote MySQL server.
- You can download MySQL from the official [MySQL website](#).

3. MySQL Workbench (Optional)

- **MySQL Workbench** is an optional tool to manage and interact with your MySQL database. It provides a graphical interface to execute queries, design databases, and more.
- You can download it from [here](#).

4. IDE (Integrated Development Environment)

- The system is developed using **NetBeans IDE**, but you can use other Java IDEs such as **Eclipse** or **IntelliJ IDEA**.
- You can download **NetBeans** from [here](#).

Steps to Set Up

Follow these steps to set up and run the **University Management System**:

1. Clone or Download the Project Files

- Download the project files from the repository or unzip the provided project files on your local machine.
- Ensure that you have the source code files, including the Java source files, and the necessary libraries (JAR files).

2. Set Up MySQL Database

- Launch **MySQL Workbench** or connect to your MySQL server using the command line.
- Create a new database (e.g., university_management_system):
- `CREATE DATABASE university_management_system;`
- Import the provided **SQL schema** into the MySQL database. This will create the necessary tables and relationships:
- `USE university_management_system;`
- `SOURCE path_to_schema.sql;`

3. Configure Database Connection

- Open the **Java project** in your IDE (NetBeans, IntelliJ, etc.).
- Locate the Conn.java class in the project (or wherever the database connection details are stored).
- Edit the database connection string to match your local MySQL setup. Update the following fields:
 - **Database URL:**
jdbc:mysql://localhost:3306/university_management_system
 - **Username:** root (or your MySQL username)
 - **Password:** The password for your MySQL user
- Example:
- public static final String DB_URL =
"jdbc:mysql://localhost:3306/university_management_system";
- public static final String USER = "root";
- public static final String PASS = "your_password";

4. Install Required Libraries

- Ensure that the required libraries (JAR files) are added to the project. These include:
 - **mysql-connector-java.jar** for database connectivity.
 - **jcalendar-tz-1.3.3-4.jar** for calendar components.
 - **rs2xml.jar** for displaying MySQL result sets in a JTable.
 - **flatlaf-3.7.jar** for the user interface look and feel.
- If you are using NetBeans, you can add these JAR files to the project by right-clicking on the project, selecting **Properties**, and adding them under the **Libraries** section.

5. Compile and Run the Application

- Once the project is configured and the database is set up, you can compile and run the application from your IDE.
- Run the Login.java class to start the application.
 - On successful login, the admin will be directed to the main dashboard.

6. Test the System

- After running the system, test all functionalities (e.g., adding/updating students/teachers, entering marks, leave management, etc.) to ensure everything is working as expected.
- Ensure that data is being stored and retrieved correctly from the database.

Running the Application

After successfully setting up the system, follow these steps to use the **University Management System**:

1. Launch the Application:

- Open your IDE and run the Login.java class to start the system.

2. Admin Login:

- Enter the **username** and **password** to log in to the system. The default credentials can be set in the database or configuration file.
- Upon successful login, the admin is redirected to the main dashboard where all functionalities are accessible.

3. Access Functionalities:

- Use the sidebar menu to navigate through different sections of the system, including:
 - **Student/Teacher Management**
 - **Marks Entry**
 - **Leave Management**
 - **Fee Structure and Fee Form**
 - **Examination Details**

4. Generate Reports:

- Generate reports for student marks, leave records, or fee details as needed.

Troubleshooting

- **Error: Unable to connect to MySQL Database**

- Ensure that the **MySQL server** is running.
- Double-check the database connection details (username, password, and database URL) in Conn.java.
- Ensure that the MySQL JDBC driver (mysql-connector-java.jar) is properly added to the project.
- **Error: Incorrect Data Format**
 - Ensure that all input fields in the forms are validated properly. For example, ensure that marks are within the allowed range (e.g., 0-100).
 - Make sure that required fields (e.g., student name, marks) are not left blank.
- **Error: UI Components Not Displaying Correctly**
 - Make sure that **Java Swing** components are properly laid out, and all necessary libraries (e.g., flatlaf-3.7.jar) are added for the UI styling.

User Manual

This section provides detailed instructions on how to use the **University Management System**. It explains the various functionalities available to the user, the steps to perform tasks, and how to navigate through the system. The user manual is intended for **administrators** (admins) who have full access to the system's features.

Admin Login

The **Admin Login** page is the first screen the user encounters when accessing the system. It ensures that only authorized personnel can access and manage the data.

- **Steps to Log In:**
 1. Open the **University Management System**.
 2. The **Admin Login** page will appear with fields for **username** and **password**.
 3. Enter the **username** and **password** assigned to the admin.
 4. Click the **Login** button to proceed.
 5. If the credentials are correct, you will be redirected to the **Main Dashboard**. If they are incorrect, an error message will be displayed.

Navigating the System

Once logged in, the admin is presented with the **Main Dashboard**, which includes a sidebar for easy access to different sections of the system.

- **Sidebar Menu:**
 - The sidebar contains options for navigating to different functionalities such as:
 - **Add/Update/View Student/Teacher Details**
 - **Marks Entry**
 - **Leave Management**
 - **Fee Structure & Fee Form**
 - **Examination Details**
 - **About Section**

Adding/Updating/View Student/Teacher Details

The admin can **add**, **update**, or **view** student and teacher records through this section.

- **Steps to Add a Student or Teacher:**
 1. In the **Main Dashboard**, click on the **Add Student** or **Add Teacher** option from the sidebar.
 2. A form will appear, requesting the following details:
 - **Name** (Required)
 - **Department** (For students)
 - **Subject** (For teachers)
 - **Phone Number**
 - **Email**
 3. Fill in the details and click **Save** to add the record to the system.
 4. If the record is successfully added, a confirmation message will appear.
- **Steps to Update a Student or Teacher:**
 1. In the **Main Dashboard**, click on **View Students** or **View Teachers**.

2. A table displaying all students or teachers will appear.
 3. Select a record to update and click the **Edit** button next to the record.
 4. Modify the required fields and click **Save** to update the record.
- **Steps to View Student/Teacher Details:**
 1. From the **Main Dashboard**, click on **View Students** or **View Teachers**.
 2. A table with all student or teacher records will appear.
 3. You can **filter**, **sort**, or **search** through the records using the available options.

Marks Entry

This functionality allows the admin to enter and manage students' marks.

- **Steps to Enter Marks:**
 1. From the **Main Dashboard**, click on **Enter Marks**.
 2. A list of students will appear. Select a student from the list.
 3. The system will display a form to enter marks for various subjects.
 4. Enter the marks for each subject, ensuring they are within the valid range (0 to 100).
 5. Click **Submit** to save the marks in the database.
- **Viewing Marks:**
 1. From the **Main Dashboard**, click on **View Marks**.
 2. A table will display the marks of all students by subject.
 3. You can search or filter the table to view specific students' marks.

Leave Management

The system allows the admin to manage **leave records** for students and teachers.

- **Steps to Add Leave Record:**
 1. In the **Main Dashboard**, click on **Manage Leaves**.
 2. Select whether you are adding a leave record for a **Student** or a **Teacher**.

3. Enter the **leave type**, **leave date**, and **reason** for the leave.
 4. Click **Save** to add the leave record.
- **Viewing Leave Records:**
 1. From the **Main Dashboard**, click on **View Leave Records**.
 2. A table will display all leave records, showing the student/teacher name, leave type, date, and reason.
 3. You can **search**, **sort**, or **filter** the leave records based on different attributes (e.g., student name, leave type).

Fee Structure & Fee Form

The admin can manage the **fee structure** and generate **fee forms** for students.

- **Steps to Update Fee Structure:**
 1. In the **Main Dashboard**, click on **Fee Structure**.
 2. A form will appear with fields for the department and fee amount.
 3. Update the fee details and click **Save** to update the fee structure.
- **Generating Fee Forms:**
 1. In the **Main Dashboard**, click on **Generate Fee Form**.
 2. Select the **student** for whom the fee form needs to be generated.
 3. The system will generate a fee form with the student's details and the fee structure.
 4. You can then **print** or **save** the fee form.

Examination Details

The admin can add, view, or modify examination-related details.

- **Steps to Add Examination Details:**
 1. In the **Main Dashboard**, click on **Examination Details**.
 2. A form will appear asking for **subject**, **exam date**, and **exam time**.
 3. Fill in the details and click **Save** to add the examination record.

- **Viewing Examination Details:**

1. From the **Main Dashboard**, click on **View Examination Details**.
2. A table will display all examination records, including subject, date, and time.
3. You can **search** or **filter** the exam details.

About Section

The **About Section** provides information about the project, including the names and roles of the development team members.

- **Accessing the About Section:**

1. From the **Main Dashboard**, click on the **About** option in the sidebar.
2. The **About Page** will display the list of team members, their roles, and any additional information about the project.

Table 1 Manual Project Screenshots

Module	Frame Title	Screenshot Slot (GUI)
Splash	University Management System	 A screenshot of the University Management System splash screen. It features a 3D rendering of a large, multi-story university building with a clock tower. The text 'UNIVERSITY MANAGEMENT SYSTEM' is overlaid in the center. The foreground is filled with a field of colorful flowers.
Login	Login form	 A screenshot of the login form for the University Management System. It shows a 3D rendering of the same university building as the splash screen. Overlaid on the image is a login form with fields for 'Username' and 'Password', and 'Login' and 'Cancel' buttons. A small icon of a person is visible on the right side of the form.

project	Dashboard menu	
Add Student	New Student Entry	
Add Faculty	New Faculty Entry	

Student Leave	Student Leave Form	
Faculty Laeve	Fcaulty leave Form	
Enter Marks	Examination: Enter Marks	

Examination detail

check result

Check Result

Result

Back

name	name	rollno	dob	address	phone	email	class_x	class_y	CNO	course	branch
adrfgh	dflgh	15337...	Mar 19	adrfgh	03456...	adrfgh...	55	75	09875...	B.Tech	Comp...
ad	adrf	15336...	Dec 3	ghtr	23245...	kyghtr	23	443	12345...	B.Tech	Comp...

Marks

Show result

CUI SAHIWAL CAMPUS

Result of Examination 2024

Roll Number: 15336287

Semester: 1st Semester

Subject	Marks
physics	90
biology	80
chemistry	90
oop	70
dse	60

Back

View Student Info

Student Details

Search by Roll Number: 15336287

Search

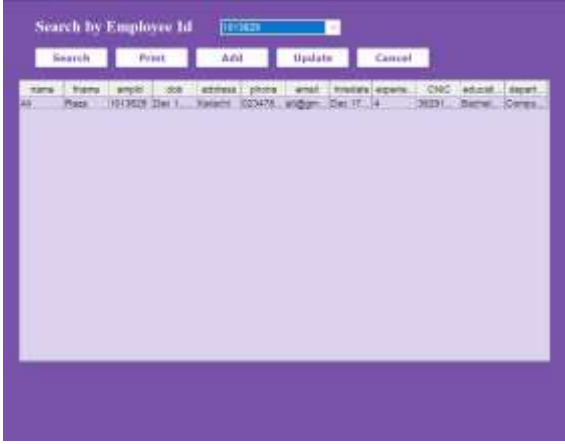
Print

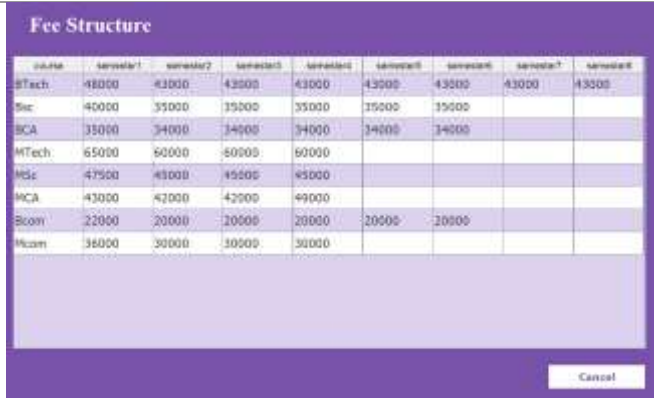
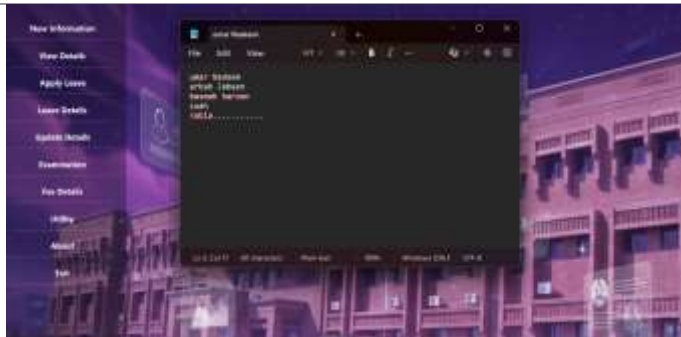
Add

Update

Cancel

name	name	rollno	dob	address	phone	email	class_x	class_y	CNO	course	branch
adrfgh	adrfgh	153378	Mar 19	adrfgh	034567	adrfgh	55	75	09875	B.Tech	Compu...
ad	adrf	153383	Dec 3	ghtr	23245	kyghtr	23	443	12345	B.Tech	Compu...

View Faculty Info	Faculty Details	
Update student	Update student info	
Update faculty	Update teacher info	

Student Fee	Student Fee Payment																																																																																		
Fee structure	Show campus fee structure	 <table><thead><tr><th>course</th><th>semester1</th><th>semester2</th><th>semester3</th><th>semester4</th><th>semester5</th><th>semester6</th><th>semester7</th><th>semester8</th></tr></thead><tbody><tr><td>BTech</td><td>48000</td><td>43000</td><td>43000</td><td>43000</td><td>43500</td><td>43000</td><td>43000</td><td>43000</td></tr><tr><td>Bsc</td><td>40000</td><td>35000</td><td>35000</td><td>35000</td><td>35000</td><td>35000</td><td></td><td></td></tr><tr><td>BCA</td><td>38000</td><td>34000</td><td>34000</td><td>34000</td><td>34000</td><td>34000</td><td></td><td></td></tr><tr><td>MTech</td><td>65000</td><td>60000</td><td>60000</td><td>60000</td><td></td><td></td><td></td><td></td></tr><tr><td>MSc</td><td>47500</td><td>43000</td><td>43000</td><td>43000</td><td></td><td></td><td></td><td></td></tr><tr><td>MCA</td><td>43000</td><td>42000</td><td>42000</td><td>49000</td><td></td><td></td><td></td><td></td></tr><tr><td>Bcom</td><td>22000</td><td>20000</td><td>20000</td><td>20000</td><td>20000</td><td>20000</td><td></td><td></td></tr><tr><td>Mcom</td><td>38000</td><td>30000</td><td>30000</td><td>30000</td><td></td><td></td><td></td><td></td></tr></tbody></table>	course	semester1	semester2	semester3	semester4	semester5	semester6	semester7	semester8	BTech	48000	43000	43000	43000	43500	43000	43000	43000	Bsc	40000	35000	35000	35000	35000	35000			BCA	38000	34000	34000	34000	34000	34000			MTech	65000	60000	60000	60000					MSc	47500	43000	43000	43000					MCA	43000	42000	42000	49000					Bcom	22000	20000	20000	20000	20000	20000			Mcom	38000	30000	30000	30000				
course	semester1	semester2	semester3	semester4	semester5	semester6	semester7	semester8																																																																											
BTech	48000	43000	43000	43000	43500	43000	43000	43000																																																																											
Bsc	40000	35000	35000	35000	35000	35000																																																																													
BCA	38000	34000	34000	34000	34000	34000																																																																													
MTech	65000	60000	60000	60000																																																																															
MSc	47500	43000	43000	43000																																																																															
MCA	43000	42000	42000	49000																																																																															
Bcom	22000	20000	20000	20000	20000	20000																																																																													
Mcom	38000	30000	30000	30000																																																																															
utility	notepad																																																																																		
utility	calculator																																																																																		

About Us	About the Project	
------------------------	---------------------------------	--

Future Enhancements

While the **University Management System** provides essential functionality for managing student, teacher, and university operations, there are many opportunities for expanding the system to meet additional requirements and enhance its capabilities. This section discusses some possible future enhancements that could improve the system's performance, scalability, and usability.

Role-Based Access Control

Currently, the system is designed with an **admin login**, which grants full access to all features. However, future versions of the system could implement **role-based access control (RBAC)** to provide different levels of access based on user roles. For example:

- **Admin:** Full access to all system functionalities.
- **Teacher:** Limited access to view and update personal records, enter marks, and manage their own leave requests.
- **Student:** Ability to view personal details, marks, and leave records.

Benefits:

- Improved security by ensuring that only authorized users can access specific system functionalities.
- A more customized user experience for different types of users.

Mobile Application

As technology continues to evolve, the use of mobile applications for managing university-related tasks is becoming increasingly popular. A **mobile version** of the University Management System could provide students, teachers, and admins with access to the system on the go.

- **Features of the Mobile App:**

- **Student View:** Access to marks, leave records, fee details, and examination schedules.
- **Teacher View:** Ability to view and update student marks, manage leave, and communicate with the administration.
- **Admin View:** Full access to manage students, teachers, leave records, and fee details.

Benefits:

- Increases accessibility by allowing users to interact with the system on their mobile devices.
- Enhances real-time communication and decision-making.

Data Analytics and Reporting

Incorporating **advanced data analytics** and **reporting** features would allow the system to generate detailed reports on various aspects of university management. Some possible reports include:

- **Student Performance Analytics:** Analyze students' performance across different subjects and identify trends (e.g., which subjects students are struggling with).
- **Fee Payment Status:** Generate reports on fee payment status for students, highlighting unpaid fees or overdue payments.
- **Leave Trends:** Analyze trends in leave requests for both students and teachers, helping the university make informed decisions regarding staffing and scheduling.

Benefits:

- Provides valuable insights into academic and financial trends.
- Supports data-driven decision-making for university administration.

Integration with Other Systems

To improve the functionality and connectivity of the University Management System, integration with other systems could be considered. For instance:

- **Learning Management System (LMS):** Integrating the University Management System with an LMS would allow for a seamless flow of academic data, including course schedules, assignments, grades, and student progress.
- **Online Payment Gateway:** Implementing an online payment gateway would enable students to pay their fees directly through the system, improving convenience and reducing administrative overhead.

Benefits:

- Streamlines university processes by connecting multiple systems.
- Enhances user convenience by allowing for online transactions and access to learning materials.

Cloud Deployment

Currently, the University Management System is designed to run on a local machine or server. However, moving to **cloud deployment** could offer significant advantages in terms of scalability, availability, and cost efficiency. Using cloud services like **Amazon Web Services (AWS)**, **Microsoft Azure**, or **Google Cloud Platform (GCP)**, the system can be hosted on the cloud, allowing for:

- **Remote Access:** Users can access the system from any device with an internet connection.
- **Scalability:** Easily scale the system to handle increasing amounts of data as the university grows.
- **Backup and Security:** Automatic backups and enhanced security features available through cloud services.

Benefits:

- Improves accessibility and ensures the system is always available.
- Reduces the need for on-premise infrastructure and maintenance.

Enhanced User Interface

While the current user interface is functional, it could be enhanced to improve the overall user experience. Some ideas for improvement include:

- **Responsive Design:** Making the interface responsive so it works seamlessly across different devices (e.g., desktops, tablets, and smartphones).
- **Improved UI Components:** Use more modern UI components and design patterns to make the interface visually appealing and easy to use.
- **Interactive Dashboards:** Create interactive dashboards with graphs and charts to provide admins with visual insights into student performance, attendance, fees, and other important metrics.

Benefits:

- Provides a more modern and user-friendly interface.
- Enhances the user experience by making the system more visually appealing and easier to navigate.

AI Integration for Predictive Analytics

Integrating **Artificial Intelligence (AI)** into the University Management System could enable predictive analytics and decision-making. For instance, the system could use AI to:

- **Predict Student Performance:** Based on historical data, AI could predict future student performance and identify students who may need additional support.
- **Automated Leave Approval:** AI could analyze leave requests and automatically approve or deny them based on predefined rules (e.g., a teacher with a high number of leave requests might need approval from a higher authority).

Benefits:

- Helps predict trends and automate decision-making, saving time and improving efficiency.
- Provides personalized recommendations for students based on their performance data.

Multi-Language Support

Expanding the system to support multiple languages would make it more accessible to users in different regions and increase its adoption in universities worldwide.

- **Features:**
 - **Language Selection:** Users can select their preferred language (e.g., English, Spanish, French, etc.) from a list.
 - **Localization:** The system interface and messages are localized to reflect the selected language.

Benefits:

- Expands the reach of the system to a global audience.
- Provides a better user experience for non-English speakers.

Conclusion

The **University Management System** is a powerful and efficient tool designed to streamline the administrative processes of managing students, teachers, exams, and university-related tasks. By utilizing **Java** for backend functionality and **MySQL** for data storage, the system provides a scalable, secure, and user-friendly platform for universities to manage their academic and administrative operations.

The system's features include:

- **Admin Login and Authentication:** Ensures secure access to the system, allowing only authorized users to manage sensitive data.
- **Student and Teacher Management:** Allows for the addition, updating, and viewing of student and teacher records, improving the accuracy and accessibility of information.
- **Marks Entry and Fee Management:** Simplifies the entry of student marks and the management of fee structures, allowing for easy generation of fee forms and reports.
- **Leave Management:** Tracks and manages leave records for both students and teachers, ensuring adherence to university policies.
- **Examination Details:** Organizes and manages examination schedules, subject information, and exam details in a centralized location.

- **About Section:** Provides information about the project team, showcasing their roles and contributions.

The **University Management System** is built with future growth in mind, and several **enhancements** are planned to improve its capabilities, including:

- **Role-based access control** for better user management.
- Development of a **mobile application** for greater accessibility.
- **Data analytics and reporting** features for deeper insights into university operations.
- Integration with other systems, such as **Learning Management Systems (LMS)** and **online payment gateways**.
- Potential migration to the **cloud** for better scalability and remote access.

Final Thoughts

The **University Management System** offers a comprehensive solution to university administration, providing the tools necessary to manage students, faculty, and academic records effectively. By automating routine tasks and centralizing data, it improves overall operational efficiency and reduces administrative overhead. With a roadmap for future enhancements, the system has the potential to adapt to the evolving needs of educational institutions, ensuring that it remains a valuable tool for years to come.

Name Reg.No.	Umar Nadeem Fa24-bse-005	Arbab Jabeen Fa24-bse-006	Basmah Haroon Fa24-bse-007	Rabia Mukhtar Fa24-bse-021	Zuha Ishtiaq Fa24-bse-034
Documentation					
Code					
Presentation					